DG INTERNS HUB

Cybersecurity Internship Program

WEEK 7

VAPT - WEB APPLICATION TESTING WITH BURP SUITE

Name: Prakhar Gupta
Track: Cybersecurity Internship
Week: Week 7 - VAPT with Burp Suite
Submission Date: 21 Sep 2025

Table of Contents

1. Introduction **3**

2. Objective **3**

3. Lab Environment & Tools **3**

4. Task 22 - Burp Suite Setup & Interception **3**

5. Task 23 - Recon & Mapping the Web Application **4**

6. Task 24 - Vulnerability Testing **6**

6.1 SQL Injection (Repeater) 6

6.2 Cross-Site Scripting (Repeater) 7

6.3 Authentication Testing (Intruder) 8

6.4 Parameter Tampering 11

7. Task 25 - Directory & Endpoint Discovery **11**

8. Task 26 - Formal Vulnerability Report **13**

9. Conclusion **14**

10. References **15**

1. Introduction

This report documents Week 7 of the VAPT practical series, focused specifically on using Burp Suite as the primary web application testing platform. Building on previous weeks' manual exploitation and automated scanning, this week introduces Burp's dedicated testing tools - Repeater, for manually modifying and resending individual requests, and Intruder, for automating repeated requests with varying payloads - alongside the Target tool for mapping an application's full attack surface, and an external directory brute-forcing tool for discovering endpoints not reachable through normal navigation.

2. Objective

The objective of this week's assignment was to perform a structured, end-to-end web application test using Burp Suite: set up interception and certificate trust for HTTPS, map the full application attack surface via the Target tool, manually test and confirm at least three classes of vulnerability using Repeater and Intruder, discover hidden endpoints through directory brute-forcing, and document the results in a formal, real-world VAPT report format.

3. Lab Environment & Tools

Component	Details
Target application	DVWA (Damn Vulnerable Web Application), self-hosted locally via Docker at http://localhost:8081, Security Level: Low
Burp Suite (Community Edition)	Intercepting proxy, Repeater, Intruder, and Target/Site map tools
Gobuster	Directory and endpoint brute-forcing tool

4. Task 22 - Burp Suite Setup & Interception

Burp Suite Community Edition was configured as an intercepting proxy on 127.0.0.1:8080, with the browser's traffic routed through it via a proxy extension. Burp's CA certificate was installed into the browser's trusted certificate store to allow interception of HTTPS traffic without certificate warnings, in addition to the plain HTTP traffic DVWA itself uses. With Intercept switched on, the DVWA login request was captured and analysed.

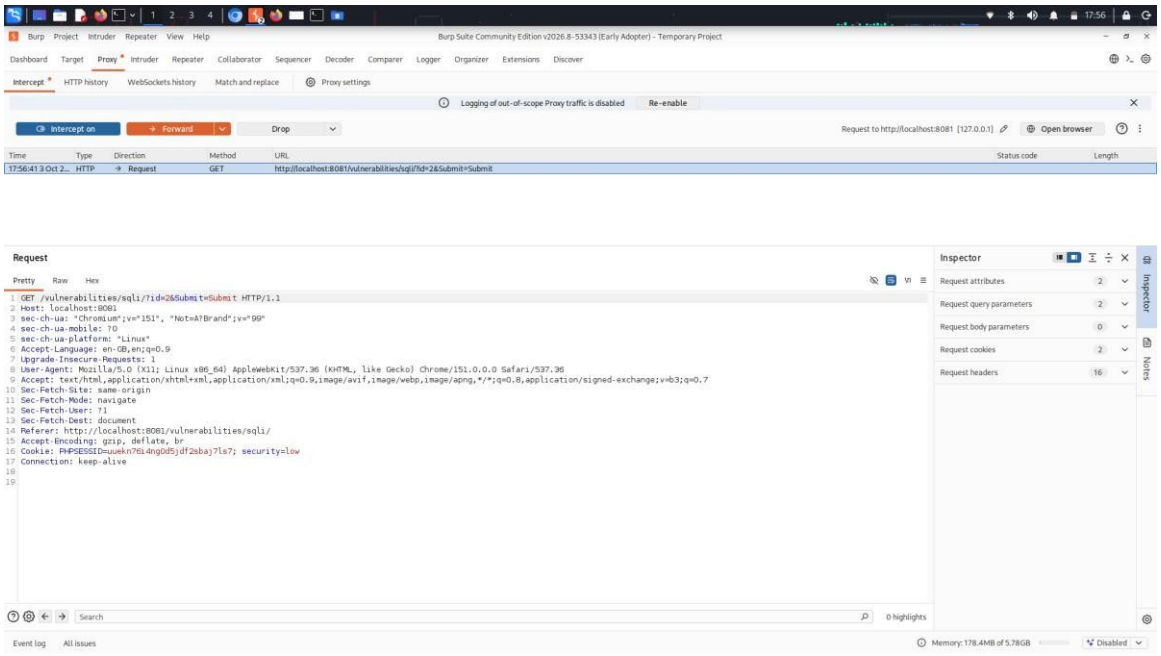


Figure 1: Burp Proxy Intercept, showing a captured GET request to the SQL Injection page with the session cookie and security level visible.

The login POST request (captured in multiple places throughout this report, e.g. Figure 9) shows the username and password transmitted as plain, unencrypted form fields (username=admin&password=...) since DVWA runs over HTTP rather than HTTPS - itself a security-relevant observation: credentials sent over an unencrypted channel can be read by anyone able to observe the network traffic.

5. Task 23 - Recon & Mapping the Web Application

With the proxy active, the full DVWA application was browsed - every page in the sidebar menu, plus the login and setup flows - allowing Burp's Target tool to automatically build a complete site map of the application as each request passed through.

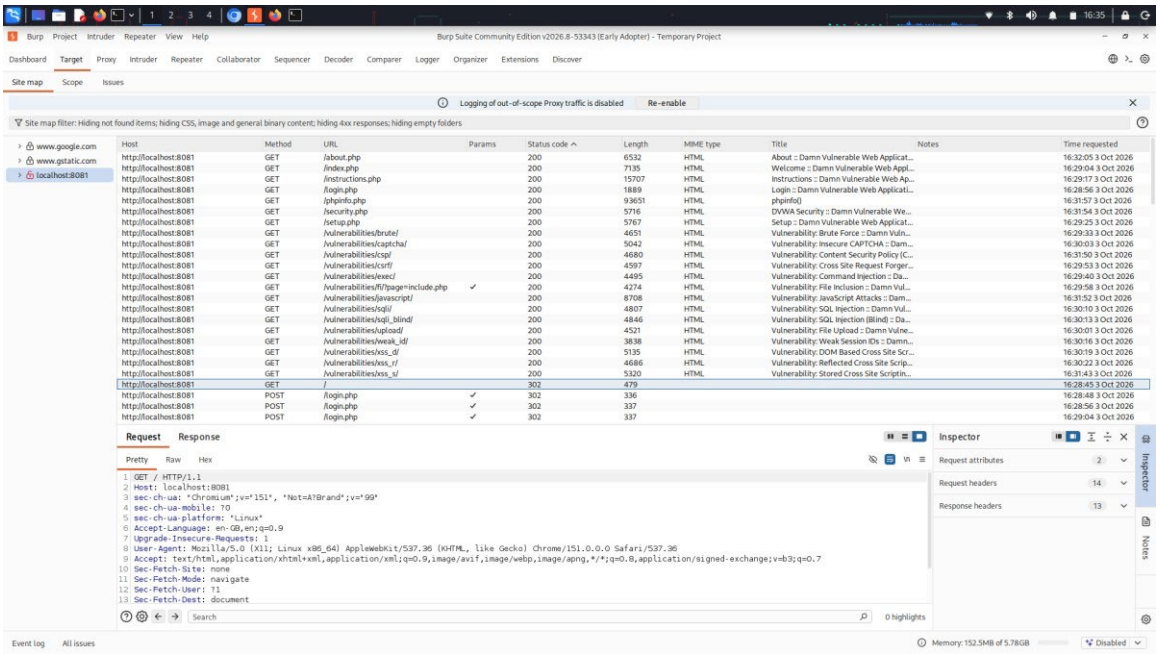


Figure 2: Burp Target - Site Map, showing the complete set of discovered DVWA endpoints, methods, status codes, and page titles.

The site map confirms the following attack surface for DVWA:

Endpoint	Method	Key Parameters
/login.php	POST	username, password, Login, user_token
/setup.php	POST	create_db, user_token
/vulnerabilities/sqli/	GET	id, Submit
/vulnerabilities/sqli_blind/	GET	id, Submit
/vulnerabilities/xss_r/	GET	name
/vulnerabilities/xss_s/	POST	txtName, mtxMessage, btnSign
/vulnerabilities/exec/	GET/POST	ip, Submit
/vulnerabilities/upload/	POST	file upload field
/vulnerabilities/csrf/	GET/POST	password_new, password_conf
/vulnerabilities/fi/	GET	page (file inclusion parameter)

This mapping exercise directly informed which endpoints and parameters were prioritised for manual testing in Task 24.

6. Task 24 - Vulnerability Testing

6.1 SQL Injection (Repeater)

The SQL Injection page's id parameter was sent to Repeater, where the payload ' OR 1=1 -- was substituted in directly, bypassing the need to resubmit the form through the browser each time.

```
Payload used:  1' OR 1=1 --
```

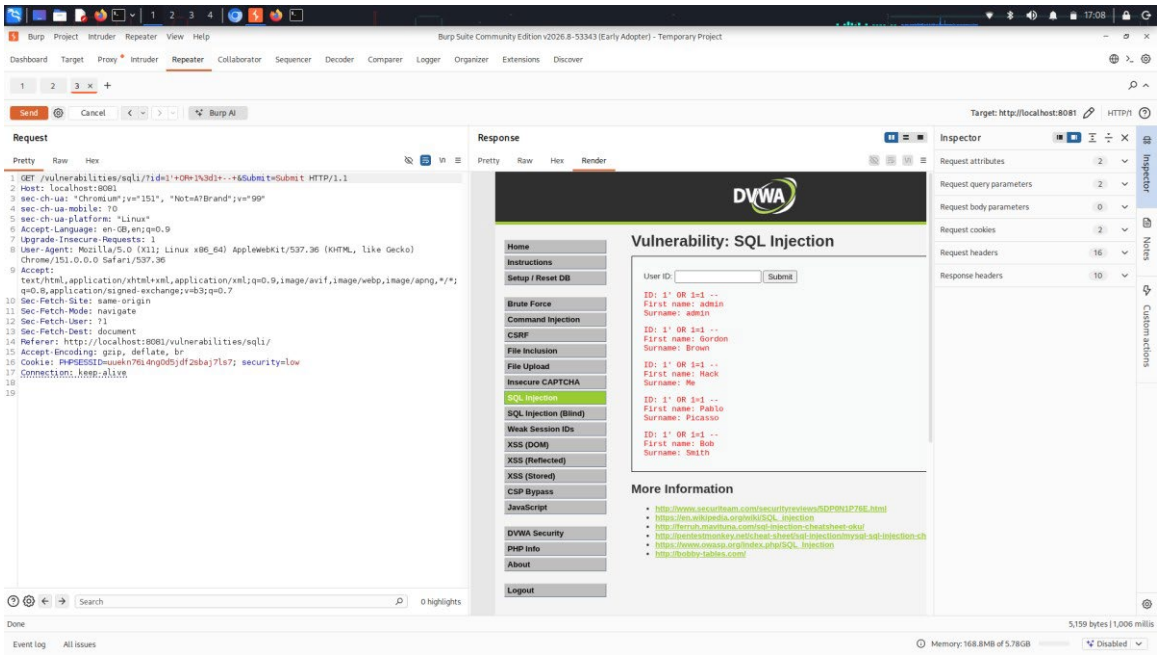


Figure 3: Repeater - the OR 1=1 payload returns every user record in the database (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) in a single response.

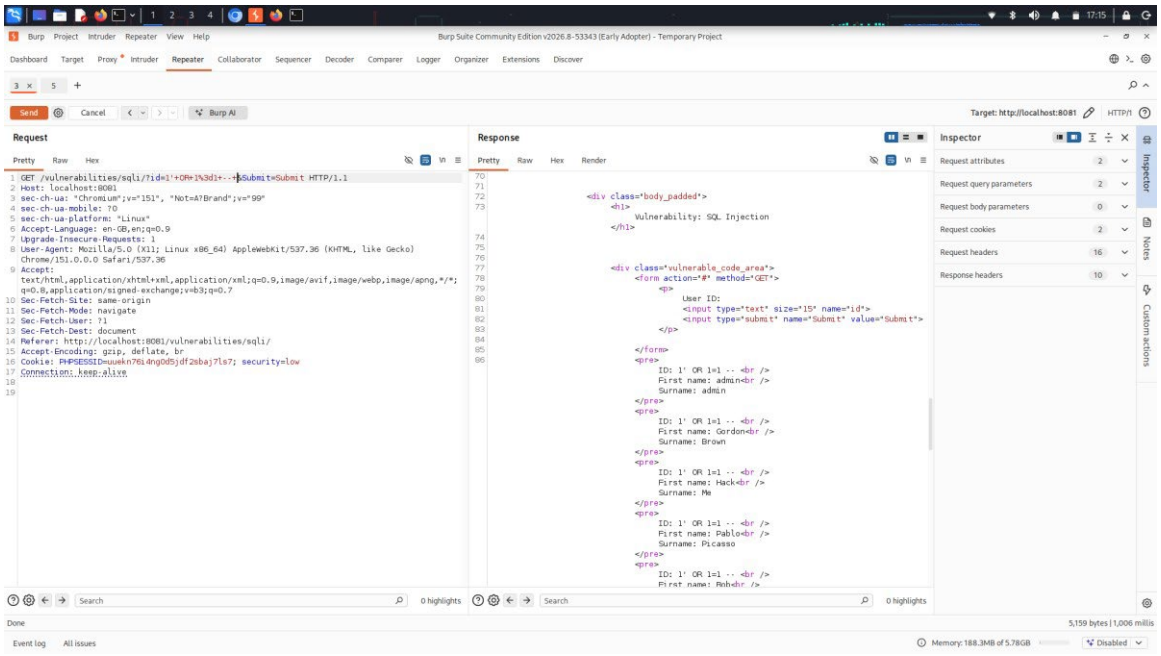


Figure 4: Repeater - full raw HTML response confirming all five user records were returned by the single injected request.

This confirms the SQL Injection vulnerability is fully exploitable via Repeater, without needing the browser form at all - demonstrating how Repeater speeds up iterative testing once an injection point has been identified.

6.2 Cross-Site Scripting (Repeater)

The XSS (Reflected) page's name parameter was sent to Repeater and modified to carry a script payload.

```
Payload used:  <script>alert(1)</script>
```

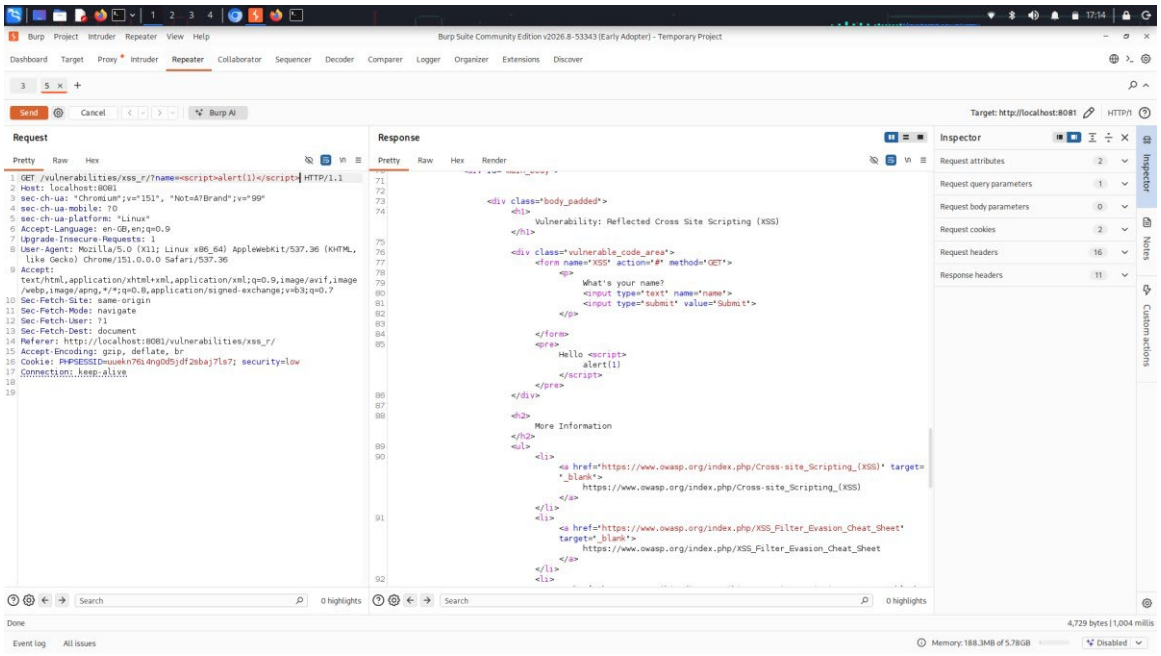


Figure 5: Repeater - the response body contains the script tag completely unescaped inside a <pre> element, confirming the payload would execute if rendered in a browser.

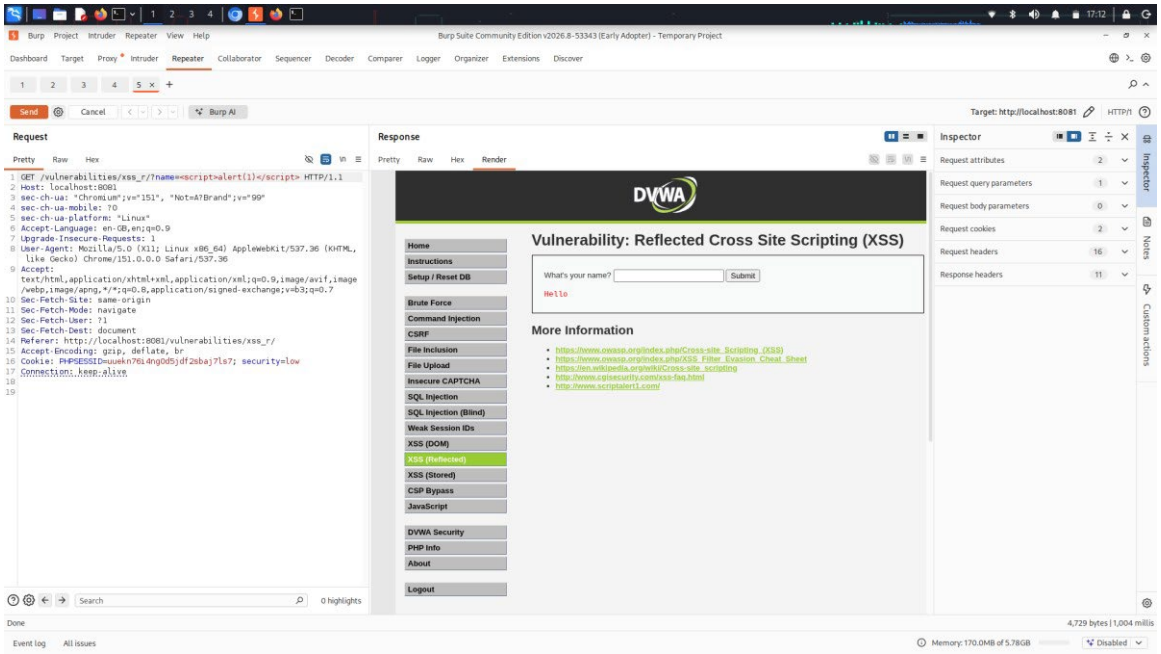


Figure 6: Repeater's built-in browser render view, confirming the page loads with the payload reflected (shown here after the script executes, leaving "Hello" on the page).

6.3 Authentication Testing (Intruder)

The DVWA login request was sent to Intruder to test for weak credential handling. The password parameter was marked as the payload position, and a short list of common passwords was loaded.

```
Payload list:  admin, 1234, admin, password
```

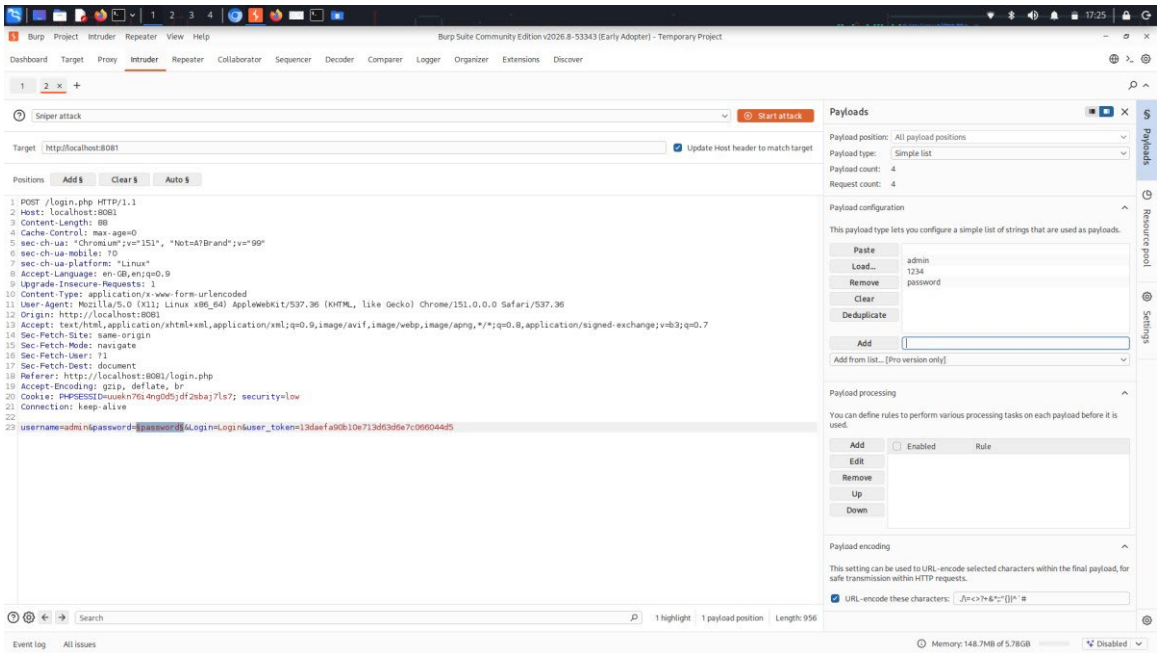


Figure 7: Intruder - Positions tab, with the password parameter marked as the single payload position (§password§) in the captured login request.

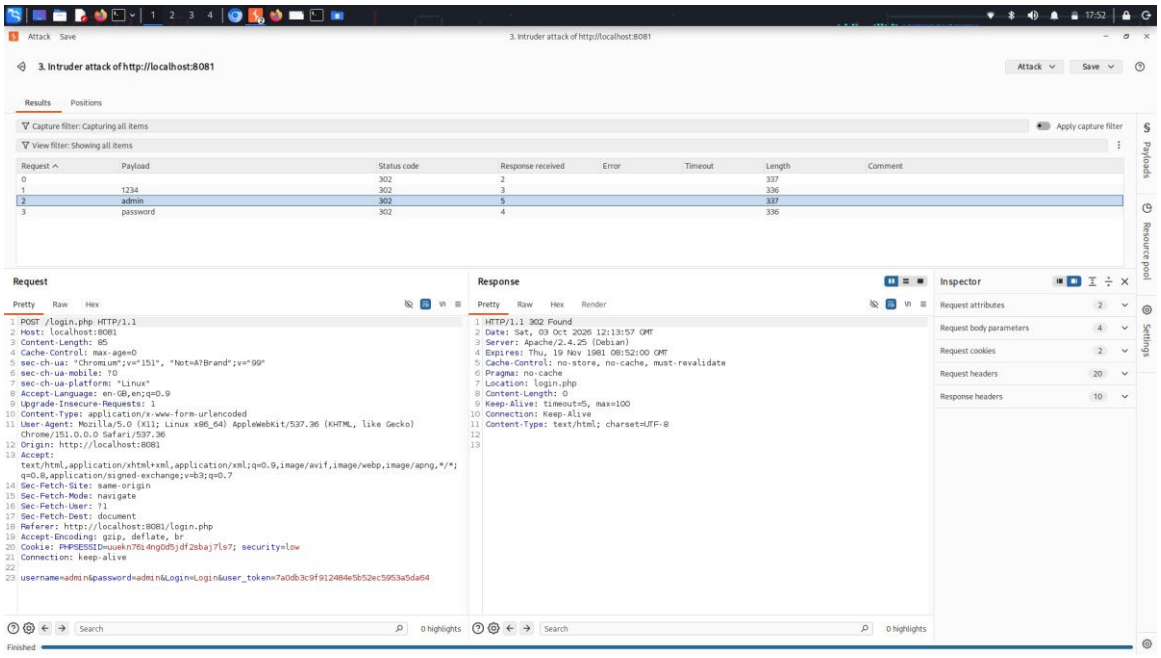


Figure 8: Intruder - Sniper attack results grid, showing status code, response length, and response time for each of the four password attempts.

All four attempts, including the correct password (password), returned the same HTTP 302 redirect back toward the login page rather than a successful authenticated redirect. This is attributable to a testing

limitation rather than a true application weakness: DVWA issues a fresh anti-CSRF user_token on every page load and invalidates it after first use, but Intruder's captured base request reused the same, alreadyconsumed token across all four attack requests, causing every attempt - including the valid credential - to be rejected uniformly. This highlights an important practical lesson for Intruder-based authentication testing: dynamic, single-use tokens must be refreshed per request (for example using Intruder's macro/session-handling features in the paid edition, or manually re-capturing the token for each attempt in the Community edition) for a credential-testing attack to produce a meaningful result.

6.4 Parameter Tampering

The id parameter on the SQL Injection page was modified in Repeater to an arbitrary, out-of-range numeric value to observe how the application handled unexpected input outside its normal 1-5 user ID range.

```
Modified value:  id=200000000
```

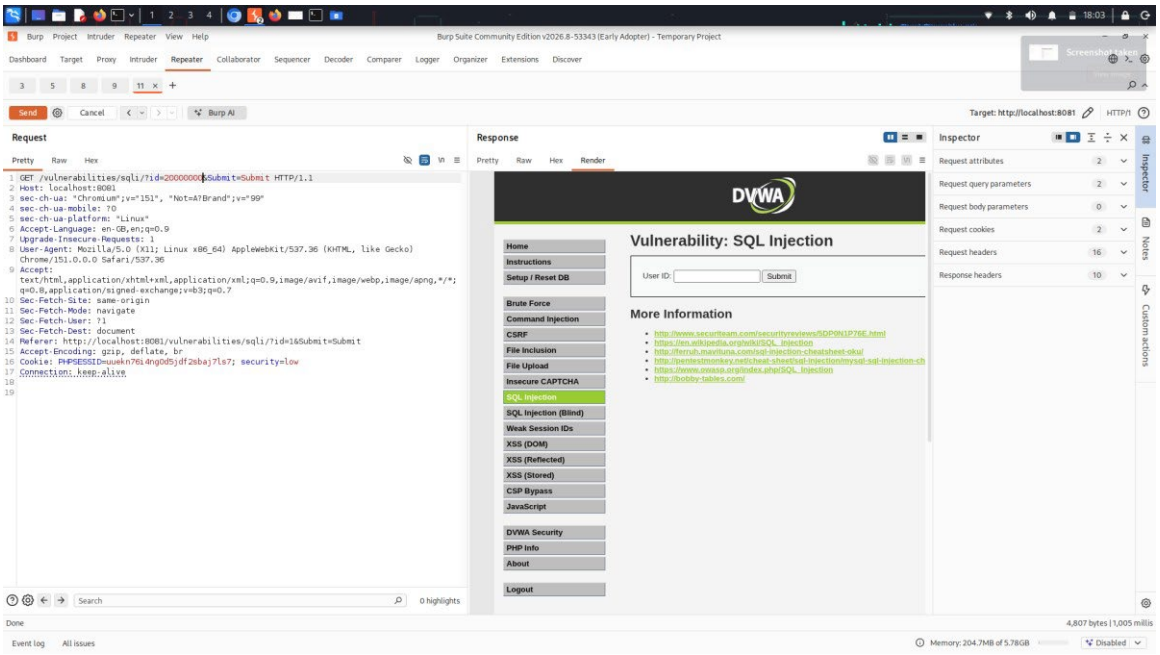


Figure 9: Repeater - request modified to an out-of-range id value; the application returned an empty result set with no error, indicating the parameter is processed without bounds validation but does not itself leak information for this specific value.

While this particular tampered value did not return additional data, the test confirms the application performs no server-side validation on the id parameter's range or type beyond what the SQL Injection testing in Section 6.1 already demonstrated - reinforcing that any value reaching this parameter is passed through to the database layer unchecked.

7. Task 25 - Directory & Endpoint Discovery

Gobuster was used to brute-force common directory and file names against the DVWA target, using the dirb common.txt wordlist.

```
gobuster dir \
  -u http://localhost:8081 \
  -w /usr/share/wordlists/dirb/common.txt
```

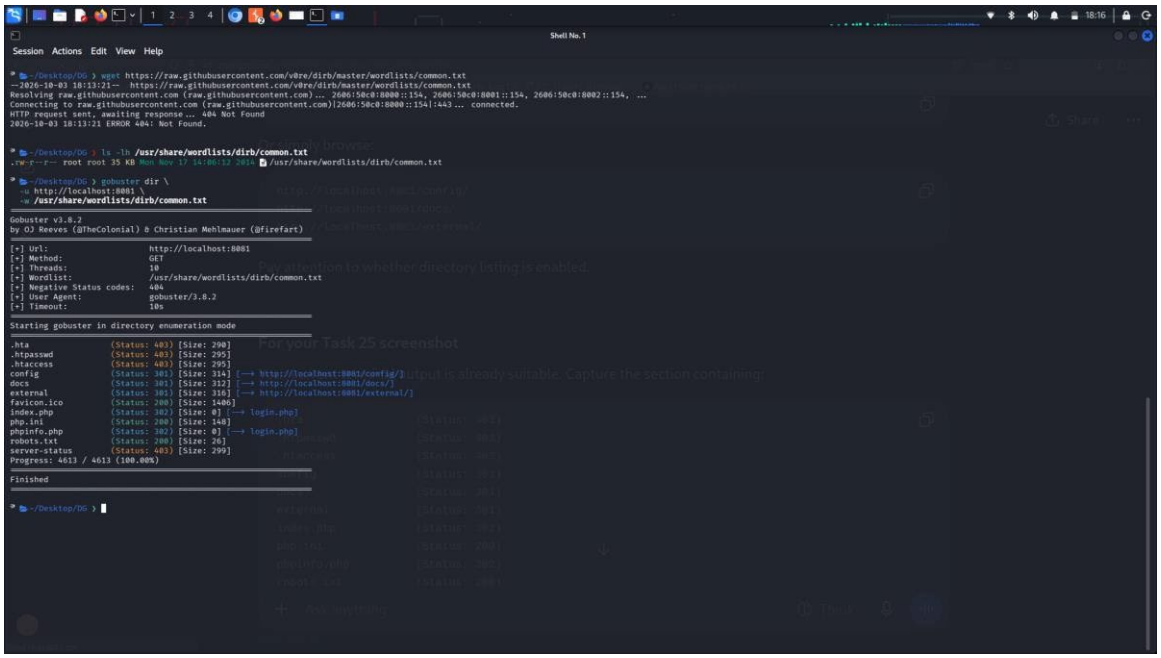


Figure 10: Gobuster directory scan results against the DVWA target, completing at 100% progress (4613/4613 requests).

The scan discovered several endpoints beyond the application's normal navigation:

Path	Status	Observation
/config/	301	Redirects to a config directory - potentially exposing database credentials if directly browsable
/docs/	301	Documentation directory
/external/	301	External/third-party library directory
/php.ini	200	PHP configuration file directly accessible
/phpinfo.php	302	Redirects to login - PHP info page present but access-gated
robots.txt	200	Present, 26 bytes
/server-status	403	Apache mod_status module enabled, though access is forbidden
.htaccess / .htpasswd	403	Present but correctly forbidden from direct access

The most notable finding is the exposed /config/ directory, since DVWA's actual database configuration file (config.inc.php, referenced during the Week 6 setup process) lives inside this path - directory bruteforcing confirms this location exists and is reachable, even though the specific config file itself was not directly listed by this wordlist.

8. Task 26 - Formal Vulnerability Report

The following three vulnerabilities are documented in standard VAPT report format, combining evidence from Task 24 and Task 25.

Vulnerability 1: SQL Injection - SQL Injection Page (id parameter)

Risk Level: Critical

Description: The id parameter on DVWA's SQL Injection page is concatenated directly into a server-side SQL query without sanitisation or parameterisation, allowing an attacker to alter the query's logic and retrieve arbitrary data from the database.

Steps to Reproduce:

- Navigate to the SQL Injection page and capture the request in Burp.
- Send the request to Repeater.
- Replace the id value with 1' OR 1=1 -- and send the request.
- Observe the response body - all five user records are returned instead of one.

Impact: An attacker can extract the complete contents of any accessible database table, including usernames, and - depending on schema - password hashes or other sensitive data, without needing any valid credentials.

Remediation: Rewrite the query using parameterised statements / prepared statements rather than string concatenation; apply least-privilege database account permissions; add server-side input validation as an additional defence-in-depth layer.

Vulnerability 2: Reflected Cross-Site Scripting - XSS (Reflected) Page (name parameter)

Risk Level: High

Description: The name parameter on DVWA's Reflected XSS page is echoed back into the HTML response without encoding or sanitisation, allowing arbitrary JavaScript supplied in the request to execute in the context of the victim's browser session.

Steps to Reproduce:

- Navigate to the XSS (Reflected) page and capture the request in Burp.
- Send the request to Repeater.
- Replace the name value with <script>alert(1)</script> and send the request.
- Observe the response body - the script tag is present, unescaped, inside the page's HTML.

Impact: An attacker who tricks a victim into clicking a crafted link containing this payload can execute arbitrary JavaScript in the victim's browser - enabling session/cookie theft, credential phishing via injected fake forms, or further redirection to malicious sites.

Remediation: HTML-encode all user-supplied output before rendering it in the page; deploy a strict Content-Security-Policy header to restrict allowed script sources as a second line of defence; validate input server-side.

Vulnerability 3: Information Disclosure via Exposed Directories

Risk Level: **Medium**

Description: Directory brute-forcing with Gobuster revealed several non-publicly-linked paths (/config/, /docs/, /external/, /php.ini) that are directly reachable, despite not being referenced anywhere in the application's normal navigation.

Steps to Reproduce:

- Run gobuster dir -u http://localhost:8081 -w /usr/share/wordlists/dirb/common.txt against the target.
- Review the results for any 2xx/3xx status codes on paths not part of the application's visible menu.
- Manually browse to the discovered paths (e.g. /config/, /php.ini) to confirm reachability and assess what each exposes.

Impact: Exposed configuration and documentation directories increase the application's attack surface and can, depending on server configuration, directly leak database credentials, internal file paths, or software version information useful for planning further attacks.

Remediation: Restrict web server access to only the directories genuinely required for the application to function; explicitly deny access to configuration, documentation, and library directories at the web server level (not just by omitting links to them); remove or relocate sensitive files such as php.ini out of the web root entirely.

9. Conclusion

This week advanced from ad-hoc manual testing into a structured, tool-driven VAPT workflow centred entirely on Burp Suite: mapping the application's full attack surface with the Target tool, iterating quickly on confirmed injection points with Repeater, automating repeated credential testing with Intruder, and supplementing browser-based discovery with external directory brute-forcing. The authentication testing exercise in particular produced a valuable lesson beyond the vulnerability findings themselves - that Burp's Intruder requires careful handling of dynamic, single-use tokens like DVWA's user_token for multi-request attacks to produce meaningful, interpretable results, a practical nuance that only becomes apparent through hands-on testing rather than reading about the tools alone. Combined with the three formally documented vulnerabilities - SQL Injection, Reflected XSS, and information disclosure via exposed directories - this week's work reflects the structure of a genuine, professional web application penetration test: systematic reconnaissance, confirmed and reproducible exploitation evidence, and riskrated findings with concrete remediation guidance for each.

10. References

- Burp Suite (PortSwigger) — https://portswigger.net/burp
- PortSwigger Web Security Academy — https://portswigger.net/web-security
- DVWA (Damn Vulnerable Web Application) — https://github.com/digininja/DVWA
- Gobuster — https://github.com/OJ/gobuster
- OWASP Top 10 — https://owasp.org/www-project-top-ten/